

INSTITUTO
FEDERAL
Santa Catarina

Resumo de matérias

Programação Frontend I

Eduardo Cardoso Oliveira, 2025

Sumário

1 Como estudar para esta disciplina? 6

2 Introdução à Programação Web 6
2.1 Frontend x Backend . . . 6
2.2 Histórico da Web . . . 6
2.3 Modelo cliente-servidor . . . 7
2.4 Ferramentas de desenvolvimento . . . 7

3 Introdução ao HTML5 7
3.1 Histórico . . . 7
3.2 O que é o HTML . . . 7
3.3 Estrutura básica de um documento HTML5 . . . 7

4 Semântica no HTML5 8

5 Elementos Genéricos, Identificação, Escape e Validação 8
5.1 Elementos genéricos: div e span . . . 8
5.2 Links: o elemento `<a>` . . . 9
5.3 Caracteres de escape . . . 9
5.4 Validação . . . 9

6 Elementos de Tabela e Mídia 9
6.1 Tabelas . . . 9
6.2 Elementos de mídia . . . 10

7 Elementos de Formulário 10
7.1 A estrutura de um formulário . . . 10
7.2 Tipos de campo `<input>` . . . 11

8 Introdução ao CSS3 11
8.1 O que é o CSS . . . 11
8.2 Histórico . . . 12
8.3 Regras de estilo . . . 12
8.4 Formas de inserir CSS no documento . . . 12
8.5 Seletores . . . 12
8.6 Herança e a cascata . . . 13

9 Aplicação do CSS3 ao HTML 13

10 Posicionando Elementos: Float e Flexbox 14
10.1 A propriedade float . . . 14
10.2 Flexbox . . . 14

11 Posicionando Elementos: Position e Grid 15
11.1 A propriedade position . . . 15
11.2 CSS Grid . . . 15

12 Responsividade com CSS3 15

12.1 O que é responsividade 15
12.2 A meta tag viewport 16
12.3 Media queries 16

13 Finalizando com Estilo: Transições, Transformações e Otimizações 16
13.1 Transições 16
13.2 Transformações 17
13.3 Propriedades personalizadas (variáveis CSS) 17
13.4 Limpando os estilos (reset) 17

14 Introdução ao JavaScript 17
14.1 Histórico 17
14.2 O papel do JavaScript 18

15 Sintaxe do JavaScript 18
15.1 Tipos de dados 18
15.2 Operadores de comparação: == x === 18
15.3 Declaração de variáveis: var, let e const 19

16 Sintaxe do JavaScript — Parte 2 19
16.1 Estruturas condicionais 19
16.2 Estruturas de repetição 19
16.3 Funções 20

17 Métodos de Arrays e Funções 21

18 Objetos JavaScript 22
18.1 O que são objetos 22
18.2 Criando objetos 22
18.3 Acessando e modificando propriedades 22

19 Manipulando o DOM 22
19.1 O que é o DOM 22
19.2 Selecionando elementos 23
19.3 Lendo e modificando atributos e conteúdo 23
19.4 Criando e removendo elementos 23

20 Eventos JavaScript 23
20.1 Formas de registrar um evento 23
20.2 O objeto evento 24
20.3 Principais tipos de eventos 24

21 Manipulando Estilos e Persistência 25
21.1 Manipulando estilos via JavaScript 25
21.2 localStorage e sessionStorage 25

22 Referências 26

Aviso importante e Isenção de responsabilidade

1. **Finalidade do documento**
   Este documento foi elaborado com o objetivo de servir como um material de apoio e revisão para os estudos das matérias do segundo semestre de Análise e Desenvolvimento de Sistemas. Todos os conteúdos aqui presentes foram retirados de anotações pessoais e resumos, compilados para facilitar a compreensão do leitor.

2. **Fontes e Direitos Autorais**
   Os resumos e anotações aqui presentes foram desenvolvidos a partir dos materiais de sala disponibilizados pelos professores (como slides, apostilas, listas de exercícios, livros, etc), anotações de aula e pesquisas. Os direitos autorais sobre os materiais originais (aulas, livros, artigos) permanecem integralmente aos seus respectivos autores e editoras. Este documento é uma obra derivada, de caráter estudantil e sem fins lucrativos.

3. **Precisão das Informações**
   Desde já, o autor não oferece nenhuma garantia de que os conteúdos aqui contidos sejam precisos e corretos, por mais que todo esforço tenha sido feito para garantir que tais informações sejam corretas. Erros, omissões ou interpretações equivocadas podem existir. O autor não se responsabiliza por quaisquer consequências, acadêmicas ou de outra natureza, que possam surgir do uso deste material.

4. **Uso Recomendado**
   Este material **NÃO SUBSTITUI DE FORMA ALGUMA** a leitura de bibliografias recomendadas por professores, presença em aulas, consulta às fontes oficiais e, principalmente, as orientações do corpo docente. Utilize este documento como uma ferramenta complementar aos seus estudos.

5. **Vínculo Institucional**
   Este é um projeto independente e não possui qualquer vínculo oficial com o Instituto Federal de Santa Catarina Campus São José, a Coordenação do Curso de Análise e Desenvolvimento de Sistemas, ou os professores das disciplinas. Não há qualquer endosso ou revisão oficial do conteúdo por parte da instituição.

Apresentação do Material

Olá, colega!

Seja bem-vindo(a) ao meu resumo de estudos! Este documento foi criado ao longo do semestre com o objetivo de centralizar e facilitar o acesso aos conteúdos que foram abordados durante as aulas do segundo semestre do curso em 2025.2 na disciplina de Programação Frontend I.

O que você vai encontrar neste documento?

- **Resumos:** Para esta disciplina trago resumo de todas as aulas do professor Adriano Lima, cobrindo os três pilares do desenvolvimento frontend: HTML5, CSS3 e JavaScript.
- **Exemplos de código:** Como essa disciplina é muito prática, trago bastante trecho de código comentado, quase sempre baseado nos próprios exemplos usados em aula.
- **Anotações e comentários:** Anotações sobre pontos que considero mais importantes ou que costumam gerar confusão (como a diferença entre `var`, `let` e `const`, ou entre `==` e `===`).
- **Documentos e bibliografias:** Quando possível, vou disponibilizar materiais, listas de exercícios e referências para auxiliar.

Como este documento está organizado?

O material segue a mesma ordem das aulas do semestre 2025.2: começamos com uma introdução geral à programação web, passamos pela estrutura (HTML5), depois pela apresentação visual (CSS3) e terminamos com o comportamento e a interatividade (JavaScript, incluindo manipulação do DOM e eventos).

Informações adicionais e contato

Este material foi feito de aluno para aluno; sugestões e colaborações são sempre bem-vindas! Se encontrar algum erro, tiver alguma sugestão para adicionar ao documento ou tirar alguma dúvida sobre qualquer matéria, fique à vontade para entrar em contato.

- **Autor:** Eduardo Cardoso Oliveira
- **E-mail Institucional:** eduardo.co22@aluno.ifsc.edu.br
- **Github:** https://github.com/EduCardoso22

Sumário

1 Como estudar para esta disciplina?

Frontend I é, provavelmente, uma das disciplinas mais ”visuais e tangíveis”do curso: tudo que você estuda, você consegue ver funcionando (ou quebrando!) na tela do navegador na hora. Por isso, o meu maior conselho aqui é simples: **escreva código, o tempo todo**. Não dá para aprender HTML, CSS ou JavaScript só lendo slide — é lendo o conceito e, na sequência, abrindo o editor de código e reproduzindo (ou modificando) o exemplo que o aprendizado realmente gruda. Sempre que a aula apresentar um atributo ou propriedade nova, teste no navegador, mude os valores e veja o que acontece.

Outro ponto importante é entender bem a **separação de responsabilidades** entre as três tecnologias centrais: o HTML define a **estrutura** do conteúdo (o ”o quê”), o CSS define a **apresentação visual** (o ”como parece”) e o JavaScript define o **comportamento** (o ”o que acontece quando...”). Entender essa separação ajuda muito a não misturar as coisas (por exemplo, evitar fazer estilização dentro do HTML, quando isso é papel do CSS). Por fim, abuse das ferramentas de desenvolvedor do navegador (`F12` ou `Ctrl+Shift+I`): inspecionar o código de páginas reais e ”brincar”com os estilos ao vivo é uma das formas mais rápidas de fixar os conceitos de CSS e JS.

2 Introdução à Programação Web

2.1 Frontend x Backend

O desenvolvimento web é tradicionalmente dividido em duas grandes frentes de trabalho:

- **Frontend** (”o lado do cliente”): cuida de tudo que o usuário vê e interage diretamente — texto, cores, imagens — usando as três tecnologias centrais: **HTML**, **CSS** e **JavaScript**. Suas principais preocupações são usabilidade, acessibilidade e responsividade.
- **Backend** (”o lado do servidor”): cuida de transformar e armazenar dados, preocupando-se com escalabilidade e segurança.

2.2 Histórico da Web

A Web, como conhecemos, nasceu de uma proposta de arquitetura de Tim Berners-Lee e Robert Cailliau, no CERN:

- **1989**: proposta da arquitetura.
- **1990**: criação do HTTP e do HTML, do primeiro navegador web (o *WorldWideWeb*), do primeiro servidor web (CERN httpd) e do primeiro site da história (`http://info.cern.ch`).
- **1991**: servidor e navegador disponibilizados publicamente.
- **1993**: lançamento do primeiro navegador gráfico, o NCSA Mosaic.
- **1994**: fundação do W3C (World Wide Web Consortium), o órgão responsável por padronizar as tecnologias web.

2.3 Modelo cliente-servidor

O **modelo cliente-servidor** é o modelo mais utilizado para interação entre computadores em rede, adotado por diversos serviços da Internet (web, e-mail, FTP, telnet) e por diversos sistemas de informação (bancos de dados, CRM, ERP). Nesse modelo, o **cliente** é quem solicita a execução de um serviço, fornece os parâmetros necessários e, ao final, exibe, armazena ou utiliza o resultado enviado pelo servidor. Já o **servidor** é quem conhece a lógica de execução do serviço e tem acesso aos recursos necessários para realizá-lo.

Na arquitetura da Web, essa relação se dá entre **navegadores web** (que acessam servidores, solicitam e exibem conteúdos, e permitem a navegação entre páginas por meio de hyperlinks) e **servidores web** (que disponibilizam conteúdo, estático ou gerado dinamicamente), tudo isso trafegando principalmente através do protocolo **HTTP** (HyperText Transfer Protocol), com o conteúdo formatado em **HTML**.

2.4 Ferramentas de desenvolvimento

Para desenvolver aplicações web, podemos usar desde um simples editor de texto (TextEdit, NotePad, Gedit), passando por editores de código mais completos (como o Sublime), até ambientes integrados de desenvolvimento (IDEs), como o Visual Studio ou o IntelliJ IDEA. Além disso, todo navegador moderno (Chrome, Safari, Edge, Firefox) possui ferramentas de desenvolvedor embutidas, acessíveis com `F12` ou `Ctrl+Shift+I`, essenciais para depurar HTML, CSS e JavaScript diretamente na página.

3 Introdução ao HTML5

3.1 Histórico

O **HTML** (HyperText Markup Language) é o padrão para a criação de páginas web, evoluindo ao longo do tempo: 1990 (HTML, com apenas 18 elementos), 1995 (HTML 2.0), janeiro de 1997 (HTML 3.2), dezembro de 1997 (HTML 4.0), 2014 (HTML5) e 2017 (HTML 5.2).

3.2 O que é o HTML

O HTML descreve a **estrutura** de uma página, sendo composto por uma série de **elementos** que dizem ao navegador como apresentar e como rotular cada parte do conteúdo. Um elemento HTML é formado por uma **tag**, podendo ter **atributos** (opcionais) e **conteúdo** (opcional):

```
<h1>Titulo</h1>
<h1 height="100px">Titulo</h1>
<img src="imagem.gif">
```

No exemplo da imagem, `img` é a tag, `src` é o atributo e ”imagem.gif”é o valor atribuído a ele.

3.3 Estrutura básica de um documento HTML5

Todo documento HTML5 segue uma estrutura mínima:

```
<!DOCTYPE html>
<html>
  <head>
    <title>Titulo da pagina</title>
  </head>
  <body>
    <h1>Meu primeiro titulo</h1>
    <p>Meu primeiro paragrafo.</p>
  </body>
</html>
```

Onde: `<!DOCTYPE html>` define o documento como HTML5; `<html>` é o elemento raiz da página; `<head>` contém informações sobre a página (que não são exibidas diretamente); `<title>` especifica o título mostrado na aba do navegador; e `<body>` define o corpo do documento, ou seja, tudo que efetivamente é exibido ao usuário.

4 Semântica no HTML5

Uma das grandes novidades do HTML5 foi a introdução de **elementos semânticos**: tags que, além de estruturar visualmente o conteúdo, também comunicam o *significado* daquele bloco, tanto para outros desenvolvedores quanto para mecanismos de busca e leitores de tela. Isso traz vantagens como otimização para mecanismos de busca (SEO), melhor acessibilidade e mais clareza no próprio código.

Os principais elementos semânticos trabalhados foram:

- `<header>`: contêiner para conteúdo introdutório, geralmente com um ou mais elementos de título.
- `<nav>`: define um conjunto de links de navegação.
- `<section>`: define uma seção em um documento. Segundo o próprio W3C, "uma seção é um agrupamento temático de conteúdo, normalmente com um título".
- `<article>`: especifica um conteúdo independente, que faz sentido por si só (como um post de blog ou uma notícia).
- `<aside>`: define algum conteúdo à parte do conteúdo principal (como uma barra lateral).
- `<footer>`: define um rodapé para um documento ou uma seção.

5 Elementos Genéricos, Identificação, Escape e Validação

5.1 Elementos genéricos: div e span

Além dos elementos semânticos, o HTML também oferece elementos **genéricos**, sem nenhum significado semântico próprio, usados puramente para agrupar conteúdo (geralmente para aplicar estilos ou comportamento via CSS/JS):

- `<div>`: elemento de bloco genérico, usado para agrupar outros elementos.
- `<span>`: elemento *inline* genérico, usado para agrupar pequenos trechos dentro de um bloco maior (como uma palavra dentro de um parágrafo).

5.2 Links: o elemento <a>

O elemento `<a>` (*anchor*) cria um hiperlink, usando o atributo `href` para indicar o destino:

```
<p>Visite a pagina do <a href="https://www.ifsc.edu.br/">IFSC</a>.</p>
```

O destino de um link não precisa ser só outra página: pode apontar para uma **âncora** dentro da própria página (usando o `id` de um elemento) ou até abrir outros programas, como o cliente de e-mail ou o discador de telefone:

```
<h2 id="c2">Segundo Conteudo</h2>
<a href="#c2">Segundo Conteudo</a>
<a href="index.html#c2">Segundo Conteudo</a>

<a href="mailto:adriano.lima@ifsc.edu.br">email</a>
<a href="tel:+01-0800-555-1212">Ligue gratis para 0800-555-1212</a>
```

5.3 Caracteres de escape

Como os caracteres `<`, `>` e `&` têm um significado especial na sintaxe do HTML (marcam o início e fim de tags), eles precisam ser ”escapados”quando queremos exibi-los como texto comum na página, usando entidades especiais:

Caractere	Significado	Entidade
<	menor que	`<`
>	maior que	`>`
&	”e”comercial	`&`
©	direito autoral	`©`

5.4 Validação

Validar um documento HTML significa verificá-lo para ter certeza de que todas as regras da linguagem foram cumpridas; documentos sem erros são considerados **válidos**. Vale a pena validar porque documentos válidos tendem a ser mais consistentes, são exibidos mais rapidamente e são mais acessíveis. A validação pode ser feita manualmente ou com o auxílio de um software validador (seja como extensão da IDE, seja online), como os disponíveis em `https://validator.w3.org` e `https://html5.validator.nu`.

6 Elementos de Tabela e Mídia

6.1 Tabelas

Uma tabela HTML é construída com o elemento `<table>`, contendo linhas (`<tr>`, de *table row*) que, por sua vez, contêm células de cabeçalho (`<th>`, de *table header*) ou células de dados comuns (`<td>`, de *table data*):

```
<table>
  <tr>
    <th>Item</th>    <th>Qtd</th>  <th>Valor diario</th>
  </tr>
  <tr>
    <td>Gorduras</td> <td>0,5</td> <td>1%</td>
  </tr>
  <tr>
    <td>Carboidratos</td> <td>24</td> <td>9%</td>
  </tr>
</table>
```

Para tabelas mais elaboradas, o HTML também oferece: `<caption>` (que deve ser o primeiro elemento dentro de `<table>`, definindo uma legenda para a tabela), `<thead>`, `<tbody>` e `<tfoot>` (que agrupam, respectivamente, o cabeçalho, o corpo e o rodapé da tabela, cada um contendo um ou mais elementos `<tr>`), e `<colgroup>`/`<col>` (posicionados no início da tabela, logo após `<caption>`, permitindo aplicar estilos a colunas inteiras).

6.2 Elementos de mídia

Para incorporar conteúdo externo ou multimídia em uma página, o HTML5 oferece diversos elementos:

- `<iframe>`: insere outra página HTML dentro da página atual, como uma ”janela”dentro do documento.
- `<object>`: carrega um arquivo (local ou de um link), podendo ser usado, por exemplo, para incorporar vídeos.
- `<video>`: insere um vídeo diretamente, sem precisar de plugins externos.
- `<audio>`: insere um arquivo de áudio diretamente.

Exemplo de uso do elemento de vídeo:

```
<video src="formatura.mp4" width="640" height="480" controls>
  Seu navegador nao suporta video HTML5. Baixe o video aqui.
</video>

<audio id="turbina" controls preload="auto">
</audio>
```

7 Elementos de Formulário

7.1 A estrutura de um formulário

Um formulário HTML é delimitado pelo elemento `<form>`, que normalmente recebe o atributo `action` (indicando para onde os dados serão enviados) e o atributo `method` (indicando como, por exemplo `GET` ou `POST`):

```
<form action="/exemplo.php" method="POST">
  <label for="login">Informe seu login:</label>
  <input type="text" name="user_login" id="login">
</form>
```

O elemento `<label>` associa um rótulo textual a um campo do formulário (ligando-se ao campo pelo atributo `for`, que deve corresponder ao `id` do campo), o que é importante tanto para a usabilidade quanto para a acessibilidade (clicar no rótulo também foca/seleciona o campo).

7.2 Tipos de campo `<input>`

O elemento `<input>` é o mais versátil do formulário, e seu comportamento muda completamente de acordo com o atributo `type`. Entre os tipos vistos em aula:

- **Texto**: `text` (texto simples), `email` (valida formato de e-mail), `search` (campo de busca), `tel` (telefone), `url` (endereço web), `password` (oculta o texto digitado).
- **Seleção**: `radio` (botão de opção única dentro de um grupo com mesmo `name`), `checkbox` (caixa de seleção independente).
- **Data e hora**: `date`, `time`, `datetime-local`, `month`, `week`.
- **Numéricos e visuais**: `number` e `range` (ambos aceitando os atributos `min`, `max` e `step` para definir os limites e o incremento), `color` (seletor visual de cor).
- **Ação**: `submit` (botão que envia o formulário).

Há também o `<textarea>` (para textos longos, com múltiplas linhas) e o `<select>` com `<option>` (uma lista suspensa de opções):

```
<input type="radio" name="idade" value="menos24" checked> Menos de 24
<input type="radio" name="idade" value="25a34"> 25 a 34

<input type="checkbox" name="musica" value="metal" checked> Metal
<input type="checkbox" name="musica" value="punkrock"> Punk Rock

<select id="banda" name="banda">
  <option value="dazaranha" selected>Da Zaranha</option>
  <option value="engenheiros">Engenheiros do Hawaii</option>
</select>
```

O atributo `placeholder` exibe um texto de exemplo dentro do campo vazio (que desaparece ao digitar), útil como dica visual para o usuário, embora não substitua o `<label>`.

8 Introdução ao CSS3

8.1 O que é o CSS

O **CSS** (Cascading Style Sheets) é o padrão para definir a apresentação visual de documentos HTML, oferecendo controles precisos de tipografia e layout, garantindo con-

sistência na formatação, reduzindo a complexidade e repetição do código, e contribuindo para sites mais acessíveis.

8.2 Histórico

- **1996**: CSS1 — instruções básicas para fontes, cores e espaçamentos (com pouco suporte dos navegadores na época).
- **1998**: CSS2 — adiciona instruções de posicionamento de elementos, estilos para outros tipos de mídia (impressa, móvel) e métodos mais sofisticados de seleção.
- **2011**: CSS2.1 — pequenos ajustes sobre o CSS2.
- **2014**: CSS3 — passa a ser implementado em módulos independentes, o que permite que a linguagem evolua de forma mais flexível.

8.3 Regras de estilo

Uma regra CSS é formada por um **seletor** (que identifica o elemento a ser afetado) e um **bloco de declaração** (contendo um conjunto de declarações para aquele seletor), onde cada declaração é um par `propriedade: valor;`:

```
h1 { color: green; }

p {
  font-size: large;
  font-family: sans-serif;
}
```

8.4 Formas de inserir CSS no documento

- **Folha de estilo externa**: um arquivo `.css` separado, importado via `<link>` no HTML ou `@import` no próprio CSS. Permite que vários documentos HTML compartilhem a mesma folha de estilo.

```
<link rel="stylesheet" href="../css/exemplo.css">
```

- **Folha de estilo incorporada**: definida dentro do elemento `<style>` no `<head>` do documento, valendo apenas para aquele documento.

8.5 Seletores

Os principais tipos de seletores trabalhados foram:

- **Elemento**: `h1 { color: green; }` afeta todos os elementos `<h1>`.
- **Grupo de elementos**: `td, th { border: 1px solid gray; }` aplica a mesma regra a vários seletores.
- **Descendente**: `li em { color: olive; }` seleciona um elemento `em` que esteja em qualquer nível dentro de um `li`.

- **Filho direto** (>): `p > em { font-weight: bold; }` seleciona apenas os `em` que são filhos *diretos* de um `p`.
- **Irmão adjacente** (+): `h1 + p { font-style: italic; }` seleciona o `p` que vem logo depois de um `h1`.
- **Irmãos subsequentes** (~): `h1 ~ h2 { font-weight: normal; }` seleciona qualquer `h2` que venha depois de um `h1`, não necessariamente logo em seguida.
- **Classe** (.): `.especial { color: orange; }` seleciona qualquer elemento com `class="especial"`.
- **ID** (#): `#intro { ... }` seleciona o único elemento com aquele `id`.
- **Atributo**: `input[type="text"] { border: 2px solid blue; }` seleciona pelo valor de um atributo.
- **Pseudo-classes e pseudo-elementos**: selecionam estados ou partes específicas de um elemento (como `:hover` para o estado de "passar o mouse").

8.6 Herança e a cascata

Algumas propriedades CSS (como as de tipografia) são **herdadas** automaticamente pelos elementos filhos, a não ser que uma regra mais específica sobrescreva esse valor. Como vários estilos (de fontes diferentes) podem entrar em conflito para o mesmo elemento, o CSS resolve isso através do chamado "efeito cascata": os pesos de cada regra são definidos por **prioridade**, **especificidade** e **ordem**.

A ordem de **prioridade**, da menor para a maior, é:

1. Regras de estilo padrão do navegador (a "folha de estilo do agente do usuário").
2. Folhas de estilo escritas pelo leitor/usuário.
3. Folhas de estilo escritas pelo autor da página.
4. Qualquer regra marcada como `!important` pelo autor.
5. Qualquer regra marcada como `!important` pelo próprio usuário (a prioridade máxima de todas).

9 Aplicação do CSS3 ao HTML

Esta aula consolida a aplicação prática dos seletores e das regras de cascata estudados, trabalhando exemplos completos de folhas de estilo aplicadas a páginas HTML reais, reforçando como escolher o seletor mais adequado para cada situação (por exemplo, usar uma classe quando o estilo se repete em vários elementos, e um id apenas quando o elemento é único na página) e como prever o resultado final de um conjunto de regras que entram em conflito.

10 Posicionando Elementos: Float e Flexbox

10.1 A propriedade float

A propriedade `float` tira um elemento do fluxo normal do documento, ”flutuando-o”para a esquerda ou direita, permitindo que outros elementos (como texto) o contornem — é a técnica clássica usada, por exemplo, para colocar uma imagem alinhada à direita de um parágrafo:

```
img {
  float: right;
}
```

Quando um elemento flutua, os elementos seguintes podem acabar se posicionando ao redor dele de forma inesperada; para evitar isso, usa-se a propriedade `clear: both;` em um elemento, forçando-o a ”descer”para abaixo de qualquer elemento flutuante anterior.

10.2 Flexbox

O **Flexbox** é um modelo de layout muito mais moderno e poderoso que o `float`, pensado justamente para organizar elementos em uma única dimensão (linha ou coluna), distribuindo espaço e alinhamento de forma flexível. Para usá-lo, o elemento **pai** recebe `display: flex;`, o que transforma automaticamente seus filhos diretos em ”itens flexíveis”.

As principais propriedades do contêiner flexível são:

- `flex-direction`: define o eixo principal do layout — `row` (padrão, da esquerda para a direita), `row-reverse`, `column` ou `column-reverse`.
- `flex-wrap`: controla se os itens devem quebrar para uma nova linha quando não cabem — `nowrap` (padrão), `wrap` ou `wrap-reverse`.
- `justify-content`: alinha os itens ao longo do **eixo principal** — valores como `flex-start` (padrão), `flex-end` e `center`.
- `align-items`: alinha os itens ao longo do **eixo transversal** (perpendicular ao principal) — `flex-start`, `flex-end`, `center`, `baseline` ou `stretch` (padrão).
- `align-content`: semelhante ao `align-items`, mas controla o espaçamento entre *linhas* inteiras de itens (só faz efeito quando há `flex-wrap` e mais de uma linha) — aceita, além dos valores anteriores, `space-between` e `space-around`.

Já nos itens flexíveis (os filhos), as propriedades mais importantes são:

- `align-self`: sobrescreve o `align-items` do pai para aquele item específico.
- `order`: define a ordem de exibição do item dentro do grupo (o padrão de todos os itens é `0`; valores menores aparecem antes).
- `flex-grow`, `flex-shrink` e `flex-basis` (combinados na propriedade abreviada `flex`): controlam, respectivamente, quanto um item pode crescer para ocupar espaço extra, quanto pode encolher quando falta espaço, e qual seu tamanho inicial antes dessas distribuições.

11 Posicionando Elementos: Position e Grid

11.1 A propriedade position

A propriedade `position` controla como um elemento é posicionado no documento, combinada com os deslocamentos `top`, `right`, `bottom` e `left`:

- `relative`: o elemento é posicionado em relação à sua posição normal no fluxo do documento, mas o espaço original dele continua reservado.
- `absolute`: o elemento é removido do fluxo normal e posicionado em relação ao seu ancestral posicionado mais próximo (ou seja, o primeiro elemento "pai"com `position` diferente de `static`).
- `fixed`: o elemento é posicionado em relação à janela do navegador (*viewport*), permanecendo fixo na tela mesmo que a página role.

Quando elementos se sobrepõem, a propriedade `z-index` define qual fica "por cima": quanto maior o valor, mais próximo da frente o elemento aparece.

11.2 CSS Grid

O **CSS Grid** é um modelo de layout pensado para organizar elementos em **duas dimensões** (linhas e colunas simultaneamente), sendo mais adequado que o Flexbox para estruturas de página inteiras. O elemento pai recebe `display: grid;` e define a malha com:

```
.container {
  display: grid;
  grid-template-rows: 100px 400px 100px;
  grid-template-columns: 200px 500px 200px;
  grid-gap: 20px 20px;
}
```

Uma forma bastante prática de organizar um layout é através da propriedade `grid-template-area` que permite "desenhar"visualmente a disposição das áreas da página usando nomes, e depois associar cada elemento filho a uma dessas áreas com `grid-area`:

```
#um { grid-area: header; }
#dois { grid-area: ads; }
#tres { grid-area: main; }
#quatro { grid-area: links; }
#cinco { grid-area: footer; }
```

12 Responsividade com CSS3

12.1 O que é responsividade

Um site responsivo é aquele projetado para ser usado confortavelmente em qualquer dispositivo — desktop, tablet ou celular — adaptando automaticamente seu layout ao tamanho da tela disponível. Isso é especialmente importante hoje, dado o crescimento constante do uso de internet por dispositivos móveis em relação aos desktops.

12.2 A meta tag viewport

O primeiro passo para um site responsivo é configurar a *viewport* no `<head>` do HTML, instruindo o navegador a usar a largura real do dispositivo (em vez de simular uma tela de desktop e depois reduzir a página):

```
<meta name="viewport" content="width=device-width, initial-scale=1.0">
```

12.3 Media queries

As **media queries** (`@media`) são o principal recurso do CSS para aplicar estilos diferentes de acordo com características do dispositivo ou da tela, seguindo o formato geral:

```
@media tipo and (propriedade: valor) {
  /* regras que se aplicam quando o dispositivo e a tela atendem o criterio */
}
```

Entre os **tipos** de mídia estão `screen` (telas), `print` (impressão) e `all` (qualquer mídia). Entre as **propriedades** mais usadas estão `width`/`height` (com suas variantes `min-width`, `max-width`, `min-height`, `max-height`) e `orientation` (`landscape` ou `portrait`):

```
@media screen and (orientation: landscape) { ... }

@media screen and (min-width: 40em) {
  /* estilos aplicados quando a tela tem pelo menos 40em de largura */
}
```

Os valores de largura/altura usados como ponto de corte para mudar o layout (como o `40em` do exemplo) são chamados de **breakpoints**, e devem ser escolhidos de acordo com os tamanhos de tela que fazem sentido para o conteúdo do site (e não necessariamente de acordo com modelos específicos de dispositivo).

13 Finalizando com Estilo: Transições, Transformações e Otimizações

13.1 Transições

A propriedade `transition` permite criar mudanças de estilo suaves (animadas) entre dois estados de um elemento (por exemplo, ao passar o mouse sobre um botão), em vez de uma troca instantânea. Ela é composta por quatro sub-propriedades:

- `transition-property`: qual propriedade CSS será animada (obrigatório).
- `transition-duration`: quanto tempo a transição deve demorar (obrigatório).
- `transition-timing-function`: como a velocidade da mudança se comporta ao longo do tempo (valores como `ease`, `linear`, `ease-in`, `ease-out`, `ease-in-out`).
- `transition-delay`: quanto tempo esperar antes de a transição começar.

13.2 Transformações

A propriedade `transform` permite aplicar transformações geométricas a um elemento, sem afetar o fluxo normal dos demais elementos da página: `rotate()` (girar), `translate()` (realocar, deslocar), `scale()` (redimensionar) e `skew()` (inclinar).

13.3 Propriedades personalizadas (variáveis CSS)

As **propriedades personalizadas** (popularmente chamadas de "variáveis CSS") permitem armazenar valores reutilizáveis, facilitando a manutenção do código. O nome de uma propriedade personalizada sempre começa com `--`, geralmente declarada dentro do seletor especial `:root` (que representa o elemento raiz do documento), e é usada através da função `var()`:

```
:root {
  --cor-primaria: #007bff;
  --cor-secundaria: #6c757d;
  --fonte-padrao: Raleway, sans-serif;
}

h1 {
  color: var(--cor-primaria);
  background-color: var(--cor-secundaria);
  font-family: var(--fonte-padrao);
}
```

13.4 Limpando os estilos (reset)

Como cada navegador traz sua própria "folha de estilo do agente do usuário"(os estilos padrão aplicados antes de qualquer CSS nosso), e esses estilos padrão não coincidem perfeitamente entre navegadores diferentes, é comum aplicar uma folha de estilo de "limpeza"(*reset* ou *normalize*) no início do projeto, garantindo uma base visual consistente antes de começar a estilizar de fato.

14 Introdução ao JavaScript

14.1 Histórico

O JavaScript nasceu em um contexto de busca por tornar os documentos web mais dinâmicos — a ideia original era criar uma "linguagem para as massas", que ajudasse até mesmo não programadores a criarem sites interativos. Foi criado por **Brendan Eich**, na Netscape, inspirado na linguagem Scheme. A linha do tempo é a seguinte:

- **1993**: lançamento do primeiro navegador gráfico de sucesso, o NCSA Mosaic.
- **1994**: fundação da Mosaic Communications Corporation, que lança o navegador Netscape Navigator.
- **1995**: a empresa passa a se chamar Netscape Corporation; em setembro, a linguagem é lançada como **LiveScript**; em novembro, é renomeada para **JavaScript**.

- **1996**: o JavaScript é submetido à Ecma International para padronização.
- **1997**: nasce o padrão **ECMAScript**, a especificação oficial por trás do JavaScript.

14.2 O papel do JavaScript

O JavaScript é o padrão para definir o **comportamento** de documentos HTML: ele adiciona interatividade à página, melhora a experiência do usuário, complementa o HTML e o CSS, e é capaz de acessar e manipular todos os elementos, atributos e textos do documento. Alguns exemplos clássicos de interatividade construída com JS: validação de formulários, carregamento dinâmico de conteúdo, menus de navegação responsivos, sliders e carrosséis de imagens, atualização de conteúdo em tempo real, animações e efeitos visuais, e integração com mapas e geolocalização.

15 Sintaxe do JavaScript

15.1 Tipos de dados

O JavaScript possui alguns tipos primitivos fundamentais:

- **Indefinido** (`undefined`): o valor padrão de uma variável que foi declarada mas ainda não recebeu nenhum valor. Ex: `var idade;`
- **Nulo** (`null`): representa a ausência intencional de um valor. Ex: `var idade = null;`
- **Número** (`number`): representa tanto números inteiros quanto decimais. Ex: `var idade = 20;`
- **Booleano** (`boolean`): `true` ou `false`. Ex: `var idade = true;`

Um detalhe importante é o conceito de valores ”falsy”(que o JavaScript trata como falso em um contexto booleano, mesmo sem serem o valor `false` propriamente): `null`, `undefined`, `0` e a string vazia .

15.2 Operadores de comparação: == x ===

Um dos pontos que mais gera confusão (e que é bastante cobrado) é a diferença entre os operadores de igualdade:

- `==` (”é igual a”): compara apenas o **valor**, convertendo os tipos se necessário. Ex: `"5"== 5` resulta em `true` (a string ”5”é convertida para o número 5 antes da comparação).
- `===` (”é idêntico a”): compara o valor **e** o tipo, sem nenhuma conversão. Ex: `"5"=== 5` resulta em `false` (tipos diferentes: string e number).
- `!==` (”não é idêntico a”): o oposto do `===`. Ex: `"5"!== 5` resulta em `true`.

Por conta dessas conversões implícitas e inesperadas do `==`, a boa prática (e o que normalmente se usa no dia a dia) é preferir sempre o `===` e o `!==`.

15.3 Declaração de variáveis: var, let e const

Uma declaração de variável sempre começa com `var`, `let` ou `const`, seguida do nome (que deve começar com uma letra, $ ou _), podendo ser inicializada com um valor:

```
var idade = 20;
let distancia = 150;
const versao = 1.2;
```

A diferença fundamental entre as três está no **escopo**:

- `var`: escopo de **função** ou global (ignora blocos como `if` ou `for`), uma forma mais antiga e hoje menos recomendada de declarar variáveis.
- `let`: escopo de **bloco** (respeita { }, como dentro de um `if` ou `for`), sendo a forma recomendada para variáveis que vão mudar de valor.
- `const`: também tem escopo de bloco, mas o valor atribuído **não pode ser reatribuído** depois (ainda que, no caso de objetos e arrays, as propriedades/elementos internos continuem podendo ser alterados).

16 Sintaxe do JavaScript — Parte 2

16.1 Estruturas condicionais

- `if` / `else`: permite a seleção entre dois caminhos distintos de execução, de acordo com o resultado de uma expressão lógica.
- `switch`: permite a seleção entre dois ou mais caminhos, testando o valor de uma única expressão contra vários `case`; costuma ter melhor legibilidade que uma longa cadeia de `if-else`.

```
if (n === 1) {
  alert("Voce tem 1 mensagem!");
} else {
  alert("Voce tem varias mensagens!");
}

switch (mes) {
  case 1:
    alert("janeiro");
    break;
  case 2:
    alert("fevereiro");
    break;
}
```

16.2 Estruturas de repetição

- `while`: repete um bloco **enquanto** uma condição for verdadeira, testada **antes** de cada execução.

- `do...while`: igual ao `while`, mas garante a execução do bloco **pelo menos uma vez** (a condição é testada depois).
- `for`: a estrutura clássica, combinando inicialização, condição e incremento em uma só linha.
- `for...of`: percorre diretamente os **valores** de uma estrutura iterável (como um array ou uma string), sendo mais simples que o `for` tradicional quando não precisamos do índice.

```
for (let count = 0; count < 10; count++) {
  console.log(count);
}

for (let elemento of dados) {
  console.log(elemento);
}

for (let letra of palavra) {
  console.log(letra);
}
```

16.3 Funções

O JavaScript oferece diversas formas de definir funções:

- **Declaração de função**: a forma clássica, usando a palavra-chave `function`.
- **Expressão de função**: a função é atribuída a uma variável, podendo inclusive ser anônima ou ser passada diretamente como argumento de outra função.
- **Função seta** (*arrow function*): uma sintaxe mais enxuta, muito usada em funções curtas e *callbacks*.

```
// declaracao de funcao
function somaNumeros(a, b) {
  return a + b;
}

// expressao de funcao
const quadrado = function(x) { return x*x; };

// funcao recursiva (calculo de fatorial)
const f = function fatorial(x) {
  if (x <= 1) return 1;
  else return x * fatorial(x - 1);
};

// funcao seta (arrow function)
const sum = (x, y) => { return x + y; };
```

O exemplo da função `fatorial` ilustra bem o conceito de **recursão**: a função chama a si mesma com um valor menor, até atingir um caso base ($x \leq 1$), que interrompe as chamadas recursivas.

17 Métodos de Arrays e Funções

Os arrays em JavaScript oferecem diversos métodos de alta ordem (que recebem outra função como argumento) para processar seus elementos de forma mais declarativa, sem precisar escrever laços manuais:

- `forEach()`: executa uma função para cada elemento do array, sem gerar um novo array (usado apenas pelo "efeito colateral", como somar valores em uma variável externa).
- `map()`: passa cada elemento pela função e retorna um **novo array** com os resultados, mantendo o mesmo tamanho do original.
- `filter()`: retorna um **novo array** contendo apenas os elementos para os quais a função passada retornou `true`.
- `find()`: retorna o **primeiro** elemento para o qual a função retornou `true` (ou `undefined`, se nenhum for encontrado).
- `concat()`: retorna um novo array contendo os elementos do array original seguidos dos elementos informados como argumento.
- `slice()`: retorna um subconjunto do array (sem modificar o original), a partir dos índices informados.
- `sort()`: ordena o array; sem argumentos, ordena "alfabeticamente"(convertendo os elementos para string, o que pode gerar resultados inesperados com números); para ordenar números corretamente, deve-se passar uma função de comparação.

```
let a = [3, 1, 4, 1, 5];

a.forEach(valor => { soma += valor; });
let quad = a.map(x => x * x);      // [9, 1, 16, 1, 25]
let maiores = a.filter(x => x > 2); // [3, 4, 5]
let primeiro = a.find(x => x % 5 === 0); // 5

a.concat(4, 5);      // [3,1,4,1,5,4,5]
a.slice(1, -1);       // [1,4,1]

[10, 2, 30].sort((a, b) => a - b); // [2, 10, 30], ordem crescente
[10, 2, 30].sort((a, b) => b - a); // [30, 10, 2], ordem decrescente
```

18 Objetos JavaScript

18.1 O que são objetos

Um objeto em JavaScript é uma **coleção não ordenada de propriedades**, onde cada propriedade tem um nome (string) e um valor, que pode ser tanto um valor primitivo quanto outro objeto (incluindo funções, chamadas nesse caso de **métodos**). É importante lembrar que, diferente dos tipos primitivos, objetos e arrays são manipulados **por referência**: ao atribuir um objeto a outra variável, ambas passam a apontar para o mesmo objeto na memória, e uma alteração através de uma reflete na outra.

18.2 Criando objetos

A forma mais comum e recomendada é usando a notação **literal** (entre chaves):

```
let livro = {
  "titulo": "JavaScript",
  "para": "todas as audiencias",
  autor: {
    nome: "David",
    sobrenome: "Flanagan"
  }
};
```

Também é possível criar objetos com o operador `new` (seguido do construtor correspondente), como em `new Object()`, `new Array()`, `new Date()` ou `new Map()`, mas essa forma é bem menos usada no dia a dia que os literais.

18.3 Acessando e modificando propriedades

As propriedades de um objeto podem ser acessadas tanto pelo **operador ponto** (`.`) quanto por **colchetes** (`[]`) — sendo os colchetes obrigatórios quando o nome da propriedade tem espaços ou é dinâmico (vem de uma variável):

```
console.log(livro.titulo);      // "JavaScript"
console.log(livro["titulo"]);   // "JavaScript"

autor.sobrenome = "de Assis";
livro["titulo"] = "Dom Casmurro";
```

19 Manipulando o DOM

19.1 O que é o DOM

O **DOM** (*Document Object Model*) é a representação do documento HTML como uma estrutura em árvore de objetos, que o JavaScript pode ler e modificar em tempo real — é justamente essa ”ponte”que permite ao JS alterar o conteúdo, a estrutura e o estilo de uma página já carregada no navegador.

19.2 Selecionando elementos

Os principais métodos para selecionar elementos do DOM são:

- `document.getElementById("nav")`: seleciona o único elemento com aquele `id`.
- `document.querySelector("div")`: seleciona o **primeiro** elemento que corresponde a um seletor CSS qualquer (podendo usar classes, ids, atributos, etc).
- `document.querySelectorAll(".warning")`: seleciona **todos** os elementos que correspondem ao seletor, retornando uma lista (`NodeList`).

19.3 Lendo e modificando atributos e conteúdo

- `getAttribute()` / `setAttribute()`: lê ou define o valor de um atributo qualquer do elemento.
- `classList`: propriedade que dá acesso às classes CSS do elemento, com métodos como `add()`, `remove()`, `toggle()` (adiciona se não tiver, remove se já tiver) e `contains()` (verifica se a classe está presente).
- `innerHTML`: lê ou define o conteúdo **HTML** interno de um elemento (incluindo tags).
- `textContent` / `innerText`: lê ou define apenas o conteúdo em **texto puro** do elemento, sem interpretar tags HTML.

```
const link = document.querySelector("a");
const endereco = link.getAttribute("href");
link.setAttribute("href", "http://google.com");

const botao = document.querySelector("button");
botao.classList.add("ativo");
botao.classList.toggle("ativo");
```

19.4 Criando e removendo elementos

Para criar elementos dinamicamente, usa-se `document.createElement()`, inserindo o novo elemento na página com `append()` (ou removendo um elemento existente com `remove()`):

```
const item = document.createElement("li"); // cria um item de lista
item.textContent = "Pizza";
cardapio.append(item);                    // insere no final
```

20 Eventos JavaScript

20.1 Formas de registrar um evento

Existem três formas principais de associar um comportamento JavaScript a um evento:

- **Como atributo HTML**: escrever o código diretamente no atributo `on<evento>` da tag (menos recomendado, pois mistura comportamento com estrutura).

```
<button onclick="passarImg(1)">Proxima</button>
```

- **Como propriedade do elemento**: atribuir uma função à propriedade `on<evento>` do elemento via JavaScript. Limitação: só é possível associar **um único** manipulador por evento dessa forma.

```
form.onsubmit = cadastraPagto;
link.onmouseover = e => e.preventDefault();
```

- **`addEventListener()`**: a forma recomendada e mais flexível, permitindo registrar **vários** manipuladores para o mesmo evento no mesmo elemento.

```
form.addEventListener("submit", (evento) => {
  evento.preventDefault();
  // ... trata o envio do formulario manualmente
});
```

20.2 O objeto evento

Toda função manipuladora de evento recebe automaticamente um objeto **evento**, que carrega informações sobre o que aconteceu. Um dos métodos mais usados nele é o `preventDefault()`, que cancela a ação padrão do navegador para aquele evento — por exemplo, impedir que um formulário recarregue a página ao ser enviado, para que o próprio JavaScript trate o envio.

20.3 Principais tipos de eventos

- **Mouse**: `click`, `mousedown`, `mouseup`, `mousemove`, `pointerdown`.
- **Teclado**: `keydown`, `keyup`.
- **Formulário**: `submit` (envio do formulário), `change` (mudança de valor, como em um `<select>`), `input` (a cada alteração de digitação), `focus` (campo recebeu foco).
- **Janela/documento**: `load` (quando a página e todos os seus recursos terminam de carregar) e `DOMContentLoaded` (quando o HTML foi completamente carregado e processado, sem esperar por imagens e outros recursos externos — geralmente mais rápido e mais indicado para iniciar scripts).

```
document.addEventListener("keyup", (event) => {
  if (event.key === "Escape") {
    // fecha algum modal, por exemplo
  }
});

window.addEventListener("load", () => { ... });
document.addEventListener("DOMContentLoaded", () => { ... });
```

21 Manipulando Estilos e Persistência

21.1 Manipulando estilos via JavaScript

É possível ler e alterar o estilo *inline* de um elemento diretamente pela propriedade `style`, onde os nomes das propriedades CSS com hífen são convertidos para **camelCase** em JavaScript (por exemplo, `background-color` vira `backgroundColor`):

```
imagem.style.display = "block";
imagem.style.position = "absolute";
imagem.style.left = `${x}px`;
imagem.style.top = `${y}px`;

titulo.style.fontFamily = "sans-serif";
item.style.flexDirection = "row";
```

21.2 localStorage e sessionStorage

O JavaScript oferece dois objetos de **armazenamento persistente** no navegador, ambos chamados de objetos `Storage`: os valores armazenados sempre precisam ser **strings**, e o escopo é restrito à mesma origem (mesmo domínio/protocolo/porta).

- `localStorage`: os dados persistem por tempo indeterminado e ficam disponíveis entre diferentes janelas e abas do navegador.
- `sessionStorage`: os dados persistem apenas até o navegador (ou a aba) ser fechado, sendo isolados por janela/aba.

Ambos podem ser acessados de forma direta (como propriedades do objeto) ou através de métodos específicos:

```
// acesso direto
let nome = localStorage.nomeUsuario;
if (!nome) {
  nome = prompt("Qual e o seu nome?");
  localStorage.nomeUsuario = nome;
}
delete localStorage.nomeUsuario;

// usando metodos do objeto Storage
localStorage.setItem("nomeUsuario", "Fulano");
localStorage.removeItem("nomeUsuario");
localStorage.clear();
```

Como os valores armazenados são sempre strings, para guardar estruturas mais complexas (como objetos ou arrays) é comum combinar o `localStorage` com `JSON.stringify()` (para converter o objeto em texto antes de salvar) e `JSON.parse()` (para converter o texto de volta em objeto ao recuperar).

22 Referências

Referências

[1] GOOGLE DRIVE. Programação Orientada a Objetos. Disponível em: https://drive.google.com/drive/folders/1zcvfTSnQqmXXI7rMkw1-dLhf2zeAjmF2?usp=drive_link. Acesso em: 22 de outubro de 2025.

[2] LIMA, Adriano. Material de aula — Programação Frontend I. Instituto Federal de Santa Catarina, Campus São José, 2025.